

23 – Complexity Classes Examples

ALGUNOS ALGORITMOS Y ESTRUCTURAS DE DATOS SIMPLES

En las ciencias (física, biológicas y sociales), los programadores a menudo comienzan codificando rápidamente un algoritmo simple para probar la plausibilidad de una hipótesis sobre un conjunto de datos, y luego lo ejecutan en una pequeña cantidad de datos. Si esto produce resultados alentadores, comienza el trabajo arduo de producir una implementación que pueda ejecutarse (quizás una y otra vez) en conjuntos de datos grandes. Tales implementaciones necesitan basarse en algoritmos eficientes.

Los algoritmos eficientes son difíciles de inventar. Los científicos informáticos profesionales exitosos podrían inventar un algoritmo durante toda su carrera—si tienen suerte. La mayoría de nosotros nunca inventamos un algoritmo novedoso. Lo que hacemos en cambio es aprender a reducir los aspectos más complejos de los problemas que enfrentamos a problemas previamente resueltos.

Más específicamente, nosotros:

- Desarrollamos una comprensión de la complejidad inherente del problema.
- Pensamos en cómo descomponer ese problema en subproblemas.
- Relacionamos esos subproblemas con otros problemas para los cuales ya existen algoritmos eficientes.

Ten en cuenta que el algoritmo más eficiente no siempre es el algoritmo de elección. Un programa que hace todo de la manera más eficiente posible a menudo es innecesariamente difícil de entender. A menudo es una buena estrategia comenzar resolviendo el problema a mano de la manera más directa posible, instrumentarlo para encontrar cualquier cuello de botella computacional, y luego buscar formas de mejorar la complejidad computacional de aquellas partes del programa que contribuyen a los cuellos de botella.

Algoritmos de Búsqueda

Un **algoritmo de búsqueda** es un método para encontrar un elemento o grupo de elementos con propiedades específicas dentro de una colección de elementos. Nos referimos a la colección de elementos como un **espacio de búsqueda**. El espacio de búsqueda puede ser algo concreto, como un conjunto de registros médicos electrónicos, o algo abstracto, como el conjunto de todos los enteros. Un gran número de problemas que ocurren en la práctica pueden formularse como problemas de búsqueda.

Anteriormente formulamos encontrar una aproximación a las raíces de un polinomio como un problema de búsqueda y examinamos tres algoritmos—enumeración exhaustiva, búsqueda por bisección, y Newton-Raphson—para buscar en el espacio de posibles respuestas.

Examinaremos dos algoritmos para buscar en una lista. Cada uno cumple con la especificación

```
1 def search(L, e):
2     """Asume que L es una lista.
3     Devuelve True si e está en L y False en caso contrario"""
```

El lector astuto podría preguntarse si esto no es semánticamente equivalente a la expresión de Python `e in L`. La respuesta es sí, lo es. Y si no te preocupa la eficiencia de descubrir si `e` está en `L`, simplemente deberías escribir esa expresión.

Búsqueda Lineal y Uso de Indirección para Acceder a Elementos

Python usa el siguiente algoritmo para determinar si un elemento está en una lista:

```
1 for i in range(len(L)):
2     if L[i] == e:
3         return True
4 return False
```

Si el elemento `e` no está en la lista, el algoritmo realizará $\theta(\text{len}(L))$ pruebas, es decir, la complejidad es en el mejor caso lineal en la longitud de `L`. ¿Por qué "en el mejor caso" lineal? Será lineal solo si cada operación dentro del bucle puede realizarse en tiempo constante. Esto plantea la pregunta de si Python recupera el i -ésimo elemento de una lista en tiempo constante. Dado que nuestro modelo de computación asume que buscar el contenido de una dirección es una operación de tiempo constante, la pregunta se convierte en si podemos

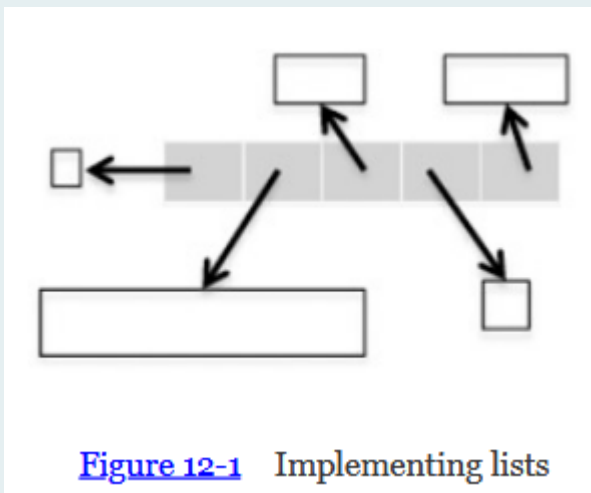
calcular la dirección del i -ésimo elemento de una lista en tiempo constante.

Comencemos considerando el caso simple donde cada elemento de la lista es un entero. Esto implica que cada elemento de la lista es del mismo tamaño, por ejemplo, cuatro unidades de memoria (cuatro bytes de 8 bits). Asumiendo que los elementos de la lista se almacenan de manera contigua, la dirección en memoria del i -ésimo elemento de la lista es simplemente $\text{start} + 4*i$, donde start es la dirección del inicio de la lista. Por lo tanto, podemos asumir que Python podría calcular la dirección del i -ésimo elemento de una lista de enteros en tiempo constante.

Por supuesto, sabemos que las listas de Python pueden contener objetos de tipos distintos a `int`, y que los mismos objetos de lista pueden contener objetos de muchos tipos y tamaños. Podrías pensar que esto presentaría un problema, pero no lo hace.

En Python, una lista se representa como una longitud (el número de objetos en la lista) y una secuencia de punteros de tamaño fijo a objetos.

❏ Implementación de listas



La región sombreada representa una lista que contiene cuatro elementos. La caja sombreada más a la izquierda contiene un puntero a un entero que indica la longitud de la lista. Cada una de las otras cajas sombreadas contiene un puntero a un objeto en la lista.

Si el campo de longitud ocupa cuatro unidades de memoria, y cada puntero (dirección) ocupa cuatro unidades de memoria, la dirección del i -ésimo elemento de la lista se almacena en la dirección $\text{start} + 4 +$

`4*i`. Nuevamente, esta dirección se puede encontrar en tiempo constante, y luego el valor almacenado en esa dirección se puede usar para acceder al i -ésimo elemento. Este acceso también es una operación de tiempo constante.

Este ejemplo ilustra una de las técnicas de implementación más importantes utilizadas en computación: **indirección**. En términos generales, la indirección involucra acceder a algo accediendo primero a algo más que contiene una referencia a la cosa buscada inicialmente. Esto es lo que sucede cada vez que usamos una variable para referirnos al objeto al cual esa variable está vinculada. Cuando usamos una variable para acceder a una lista y luego a una referencia almacenada en esa lista para acceder a otro objeto, estamos pasando por dos niveles de indirección.

Búsqueda Binaria y Explotación de Suposiciones

Volviendo al problema de implementar `search(L, e)`, ¿es $\theta(\text{len}(L))$ lo mejor que podemos hacer? Sí, si no sabemos nada sobre la relación de los valores de los elementos en la lista y el orden en el cual están almacenados. En el peor caso, tenemos que mirar cada elemento en `L` para determinar si `L` contiene `e`.

Pero supongamos que sabemos algo sobre el orden en el cual los elementos están almacenados, por ejemplo, supongamos que sabemos que tenemos una lista de enteros almacenados en orden ascendente. Podríamos cambiar la implementación para que la búsqueda se detenga cuando alcance un número mayor que aquel por el cual está buscando.

Búsqueda lineal de una lista ordenada

```
1 def search(L, e):
2     """Asume que L es una lista, cuyos elementos están en
3     orden ascendente.
4     Devuelve True si e está en L y False en caso contrario"""
5     for i in range(len(L)):
6         if L[i] == e:
7             return True
8         if L[i] > e:
9             return False
10    return False
```

Esto mejoraría el tiempo de ejecución promedio. Sin embargo, no cambiaría la complejidad del peor caso del algoritmo, ya que en el peor caso cada elemento de `L` es examinado.

Sin embargo, podemos obtener una mejora considerable en la complejidad del peor caso usando un algoritmo, **búsqueda binaria**, que es similar al algoritmo de búsqueda por bisección para encontrar una aproximación a la raíz cuadrada de un número de punto flotante. Allí nos basamos en el hecho de que hay un ordenamiento total intrínseco en los números de punto flotante. Aquí nos basamos en la suposición de que la lista está ordenada.

La idea es simple:

1. Elegir un índice, `i`, que divida la lista `L` aproximadamente por la mitad.
2. Preguntar si `L[i] == e`.
3. Si no, preguntar si `L[i]` es mayor o menor que `e`.
4. Dependiendo de la respuesta, buscar ya sea en la mitad izquierda o derecha de `L` por `e`.

Dada la estructura de este algoritmo, no es sorprendente que la implementación más directa de **búsqueda binaria** use recursión.

La función externa, `search(L, e)`, tiene los mismos argumentos y especificación que la función definida anteriormente. La especificación dice que la implementación puede asumir que `L` está ordenada en orden ascendente. La responsabilidad de asegurar que esta suposición se satisfaga recae en quien llama a `search`. Si la suposición no se satisface, la implementación no tiene obligación de comportarse bien. Podría funcionar, pero también podría fallar o devolver una respuesta incorrecta. ¿Debería `search` ser modificada para verificar que la suposición se satisfaga? Esto podría eliminar una fuente de errores, pero frustraría el propósito de usar búsqueda binaria, ya que verificar la suposición en sí misma tomaría tiempo $O(\text{len}(L))$.

Búsqueda binaria recursiva

```
1  def search(L, e):
2      """Asume que L es una lista, cuyos elementos están en
3      orden ascendente.
4      Devuelve True si e está en L y False en caso contrario"""
5
6      def bin_search(L, e, low, high):
7          #Decrementa high - low
8          if high == low:
9              return L[low] == e
10         mid = (low + high)//2
11         if L[mid] == e:
12             return True
```

```

13         elif L[mid] > e:
14             if low == mid: #nothing left to search
15                 return False
16             else:
17                 return bin_search(L, e, low, mid - 1)
18         else:
19             return bin_search(L, e, mid + 1, high)
20
21     if len(L) == 0:
22         return False
23     else:
24         return bin_search(L, e, 0, len(L) - 1)

```

Funciones como `search` a menudo se llaman **funciones envolventes** (wrapper functions). La función proporciona una interfaz agradable para el código cliente, pero es esencialmente un paso intermedio que no hace ningún cálculo serio. En su lugar, llama a la función auxiliar `bin_search` con argumentos apropiados. Esto plantea la pregunta de por qué no eliminar `search` y hacer que los clientes llamen a `bin_search` directamente. La razón es que los parámetros `low` y `high` no tienen nada que ver con la abstracción de buscar en una lista un elemento. Son detalles de implementación que deberían estar ocultos de aquellos que escriben programas que llaman a `search`.

Ahora analicemos la complejidad de `bin_search`. Mostramos en la última sección que el acceso a listas toma tiempo constante. Por lo tanto, podemos ver que excluyendo la llamada recursiva, cada instancia de `bin_search` es $\Theta(1)$. Por lo tanto, la complejidad de `bin_search` depende únicamente del número de llamadas recursivas.

Si este fuera un texto sobre algoritmos, ahora nos sumergiríamos en un análisis cuidadoso usando algo llamado relación de recurrencia. Pero como no lo es, tomaremos un enfoque mucho menos formal que comienza con la pregunta "¿Cómo sabemos que el programa termina?" Recuerda que hicimos la misma pregunta sobre un bucle `while`. Respondimos la pregunta proporcionando una **función decrementante** para el bucle. Hacemos lo mismo aquí. En este contexto, la función decrementante tiene las propiedades:

- Mapea los valores a los cuales están vinculados los parámetros formales a un entero no negativo.
- Cuando su valor es 0, la recursión termina.
- Para cada llamada recursiva, el valor de la función decrementante es menor que el valor de la función decrementante al entrar a la instancia de la función que hace la llamada.

La función decrementante para `bin_search` es `high-low`. La declaración `if` en `search` asegura que el valor de esta función decrementante sea al menos 0 la primera vez que se llama a `bsearch` (propiedad 1 de la función decrementante).

Cuando se ingresa a `bin_search`, si `high-low` es exactamente 0, la función no hace ninguna llamada recursiva—simplemente devuelve el valor `L[low] == e` (satisfaciendo la propiedad 2 de la función decrementante).

La función `bin_search` contiene dos llamadas recursivas. Una llamada usa argumentos que cubren todos los elementos a la izquierda de `mid`, y la otra llamada usa argumentos que cubren todos los elementos a la derecha de `mid`. En cualquier caso, el valor de `high-low` se reduce a la mitad (satisfaciendo la propiedad 3 de la función decrementante).

Ahora entendemos por qué la recursión termina. La siguiente pregunta es cuántas veces se puede reducir a la mitad el valor de `high-low` antes de que `high-low == 0`. Recuerda que $\log_y(x)$ es el número de veces que `y` tiene que ser multiplicado por sí mismo para alcanzar `x`. Inversamente, si `x` se divide por `y` $\log_y(x)$ veces, el resultado es 1. Esto implica que `high-low` puede ser reducido a la mitad usando división entera como máximo $\log_2(\text{high-low})$ veces antes de que alcance 0.

Finalmente, podemos responder la pregunta, ¿Cuál es la complejidad algorítmica de la búsqueda binaria? Dado que cuando `search` llama a `bsearch` el valor de `high-low` es igual a `len(L)-1`, la complejidad de `search` es $\theta(\log(\text{len}(L)))$.

Ejercicio manual

¿Por qué el código usa `mid+1` en lugar de `mid` en la segunda llamada recursiva?

Porque en el caso de que `L[mid] < e` quiere decir que `e` está en la mitad más alta de la lista ordenada, por lo cual cuando hacemos la llamada recursiva vamos incrementando el piso de esta (`low`).

Ejercicio manual de la lectura

Pregunta 1: Elija el peor caso de orden asintótico de crecimiento (límite superior e inferior) para la siguiente función. Suponga que $n = a$.

```
1 def running_product(a):
2     """ a is an int """
3     product = 1
4     for i in range(5, a + 5):
5         product *= i
6         if product == a:
7             return True
8     return False
```

El bucle ejecuta exactamente **a** iteraciones.

El peor caso es $O(n)$.

Pregunta 2: Elija el peor caso de orden asintótico de crecimiento (límite superior e inferior) para la siguiente función. Suponga que $n = \text{len}(L)$.

```
1 def tricky_f(L, L2):
2     """ L and L2 are lists of equal length """
3     inL = False
4     for e1 in L:
5         if e1 in L2:
6             inL = True
7     inL2 = False
8     for e2 in L2:
9         if e2 in L:
10            inL2 = True
11     return inL and inL2
```

Cada operación **in** en una lista toma $O(n)$ en el peor caso, y se ejecuta dentro de bucles de $O(n)$.

El peor caso es $O(n^2)$.

Pregunta 3: Elija el orden asintótico de crecimiento en el peor de los casos (límite superior e inferior) para la siguiente función.

```
1 def sum_f(n):
2     """ n > 0 """
3     answer = 0
4     while n > 0:
5         answer += n % 10
6         n = int(n / 10)
7     return answer
```

Esta función suma los dígitos de un número. El número de iteraciones depende de cuántos dígitos tiene **n**. Si **n** tiene **d** dígitos, entonces

$$d = \log_2(n) .$$

El peor caso es $O(\log n)$.